Introduction to Union Find Pattern

Union Find, also known as Disjoint Set Union (DSU), is a data structure that keeps track of a partition of a set into disjoint subsets (meaning no set overlaps with another). It provides two primary operations: **find**, which determines which subset a particular element is in, and **union**, which merges two subsets into a single subset. This pattern is particularly useful for problems where we need to find whether 2 elements belong to the same group or need to solve connectivity-related problems in a graph or tree.

Core Operations of Union-Find (Disjoint Set Union - DSU):

1. **Find:** Determine which set a particular element belongs to. This can be used for determining if two elements are in the same set.
2. **Union:** Merge two sets together. This operation is used when there is a relationship between two elements, indicating they should be in the same set.

Union-Find: A Story of Connections

Imagine a social network where everyone is initially a single user with no friends. The moment two users become friends, they form a group. If a user befriends someone from another group, the two groups merge into a larger friend circle. Union-Find helps track these friendships and circles.

How DSU Work?

1. **Initial State**: Everyone is in their own separate group. Think of each person as their own little island.
2. **Making Connections (Union)**: When two people shake hands, they now belong to the same group. If they were already in groups, their entire groups merge.
3. **Checking Connections (Find)**: Want to know if two people are connected? Union-Find can check if they're in the same group.

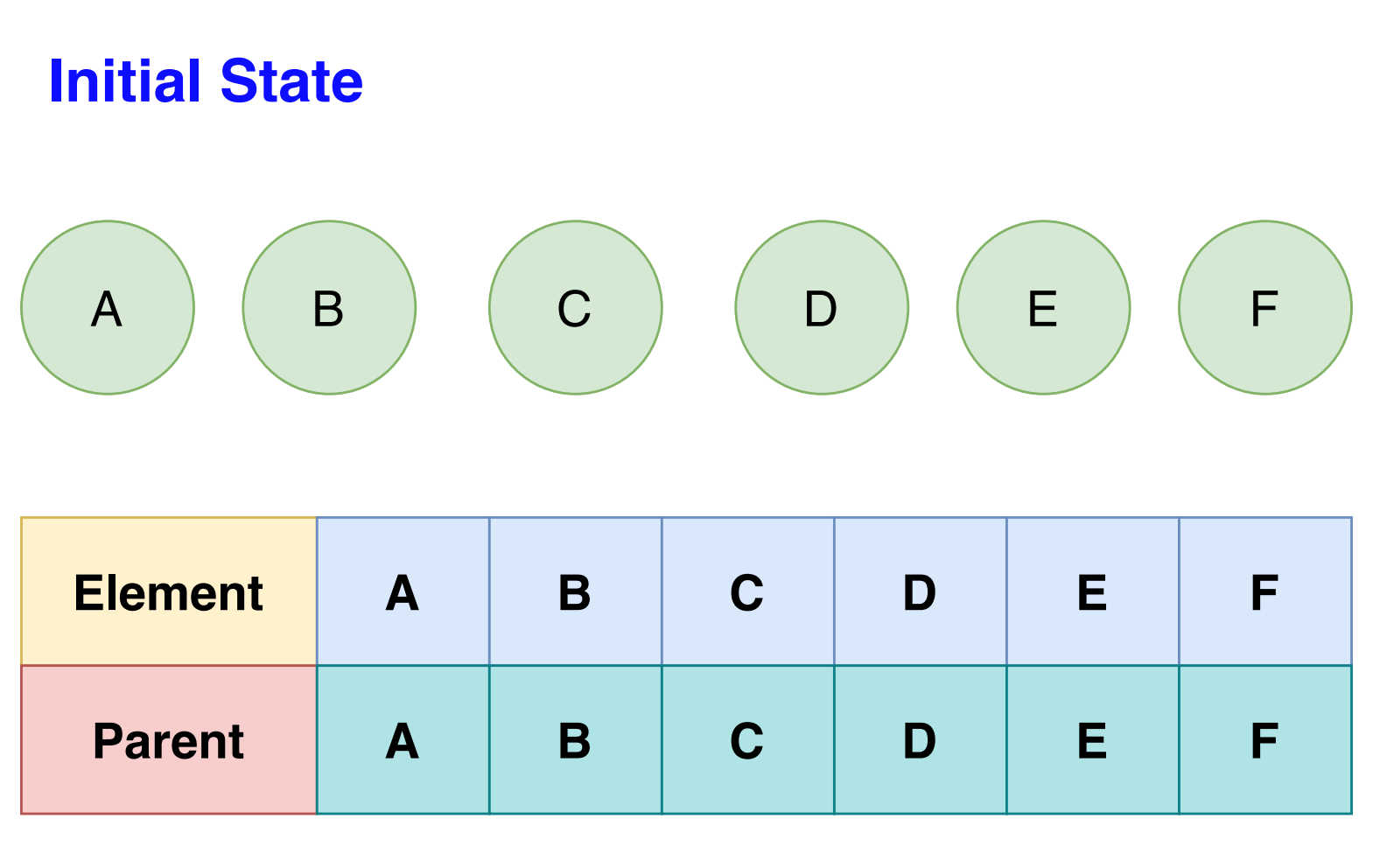
How DSU is Implemented?

- **Parents Array**: Each element has a parent. In the beginning, each element is their own parent.
- **Find Operation**: Follows the chain of parents until it reaches the root parent, which represents the group.
- **Union Operation**: Connects two groups by setting the parent of one group's root to the other group's root.

Operations on Disjoint Set Data Structures

1. Creating Disjoint Sets:

- Each element points to a parent.
- Initially, each element is its own parent, forming `n` disjoint sets.
- A "root" element is one that points to itself, identifying the set's representative.



```
# Initialize 'n' elements where each element is its own parent.
parent = [i for i in range(n)]  # 'n' is the number of elements
```

2. Find Operation:

The `Find` operation determines which set a particular element belongs to. This can be used to determine if two elements are in the same set.

In the below code:

- `parent` is an array where `parent[i]` is the parent of `i`.
- If `parent[i] == i`, then `i` is the root of the set and hence the representative.
- The `find` function follows the chain of parents for `i` until it reaches the root.

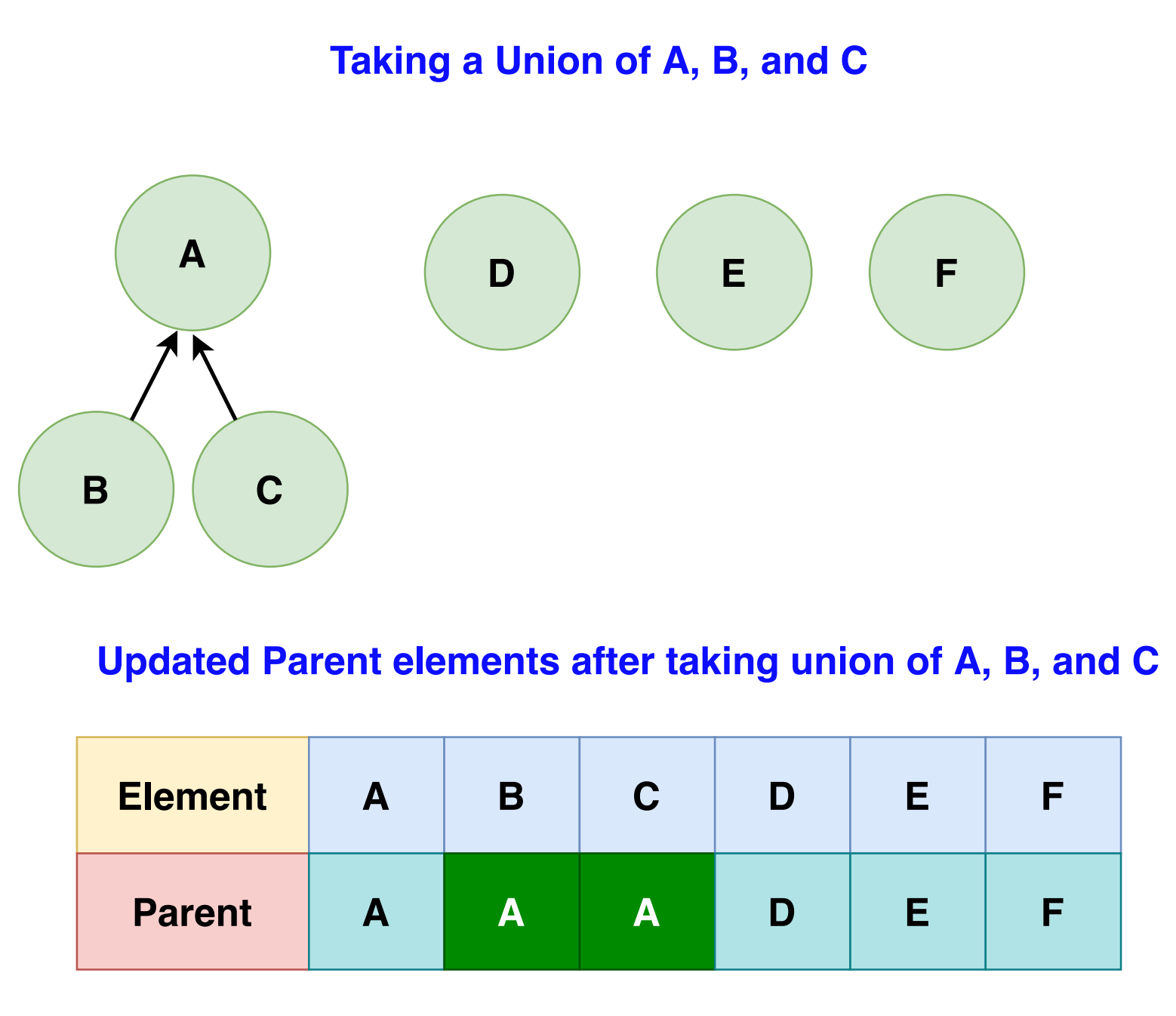
Java

```java
public int find(int i) {
  // If the element is its own parent, it's the representative of its set.
  if (parent[i] == i) {
      return i;
  }
  // Otherwise, recursively find the representative of the set.
  else {
    return find(parent[i]);
  }
}
```

Above function recursively traverses the parent array until it finds a node that is the parent of itself.

3. Union Operation:

The `Union` operation merges two disjoint sets. It takes two elements as input, finds the representatives of their sets using the `Find` operation, and finally merges the two sets.


Taking a Union of A, B, and C
A
B
C
D
E
F
Updated Parent elements after taking union of A, B, and C
Element A B C D E F
Parent A A A D E F

Java

```java
public void union(int i, int j) {
    // Find the representatives of the two sets.
    int irep = find(i);
    int jrep = find(j);

    // If they are already in the same set, return.
    if (irep == jrep) {
        return;
    }

    // Make the representative of the first set point to the representative of the secor
    parent[irep] = jrep;
}
```

Above function merges two sets by making the representative of one set the parent of the other.

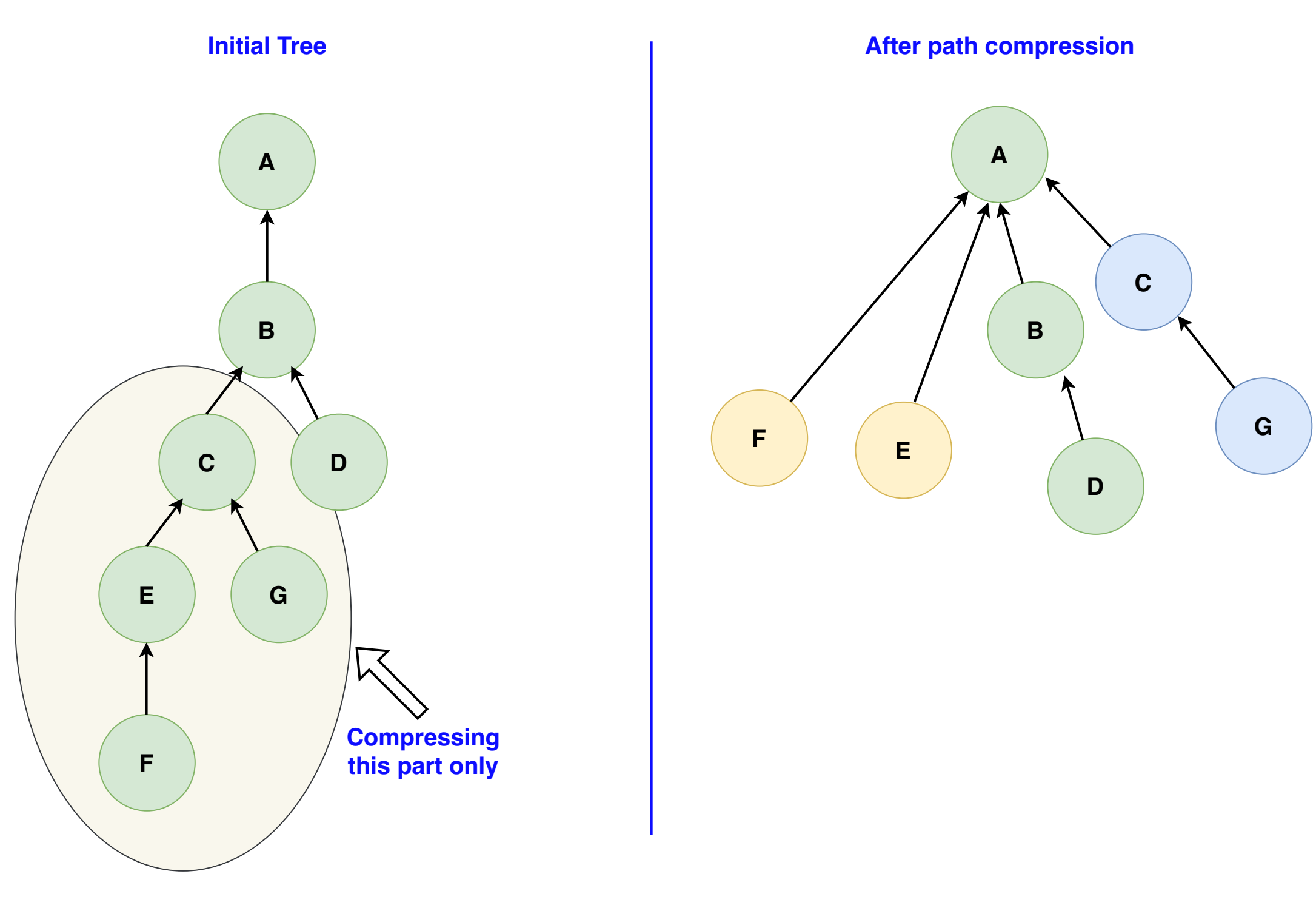
Now, we need to optimize the `find()` and `union()` methods.

Optimizations

Here are various methods for optimally perform operations on disjoint set.

1. Path Compression:

This optimization is applied during the `Find` operation. The idea behind path compression is to flatten the structure of the tree, making each member of the set point directly to the set's representative. This ensures that subsequent `Find` operations are faster and more efficient. When an element's representative is found using the `Find` operation, the element's parent is updated to directly reference the representative, thereby compressing the path.



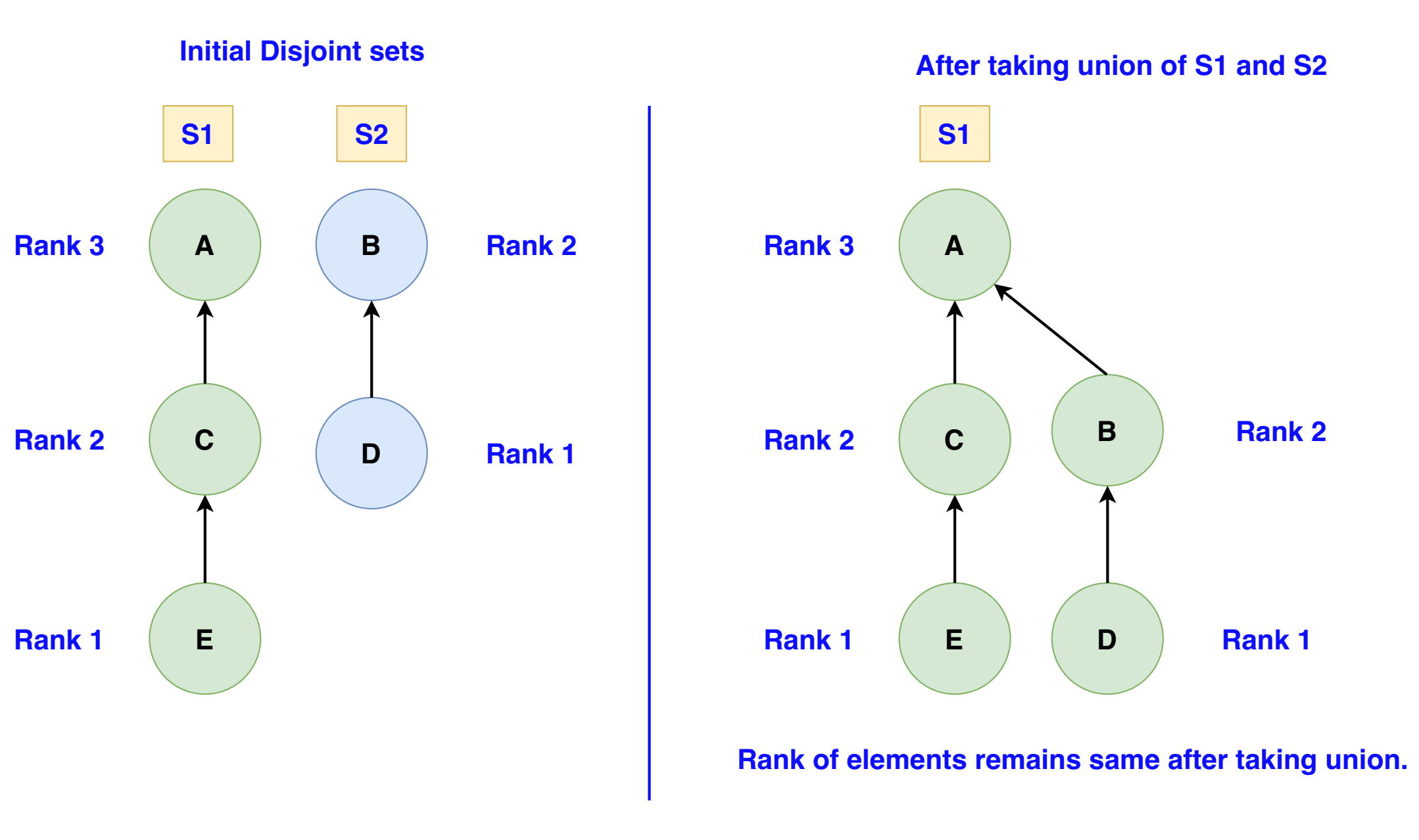
Java

```java
// Java implementation of the find function
public int find(int i) {
  // If the element is the parent of itself
  if (parent[i] == i) {
    // Then it is the representative of its set
    return i;
  } else {
    // Otherwise, recursively find the representative of the set to which 'i' belongs
    int result = find(parent[i]);
    // Path Compression: Directly connect 'i' to the representative of its set
    parent[i] = result;
    // Return the representative
    return result;
  }
}
```

This modified `Find` operation ensures that every node directly points to the representative of the set, and it makes it easy to find the representative of any two elements.

2. Union by Rank:

This optimization focuses on keeping the tree balanced during the `Union` operation. When two sets are merged, the tree with a smaller rank (or depth) is attached to the root of the tree with a larger rank. This ensures that the tree remains relatively flat, optimizing future operations. If both trees have the same rank, the rank of one of them is incremented by one, and the other tree is attached to it.



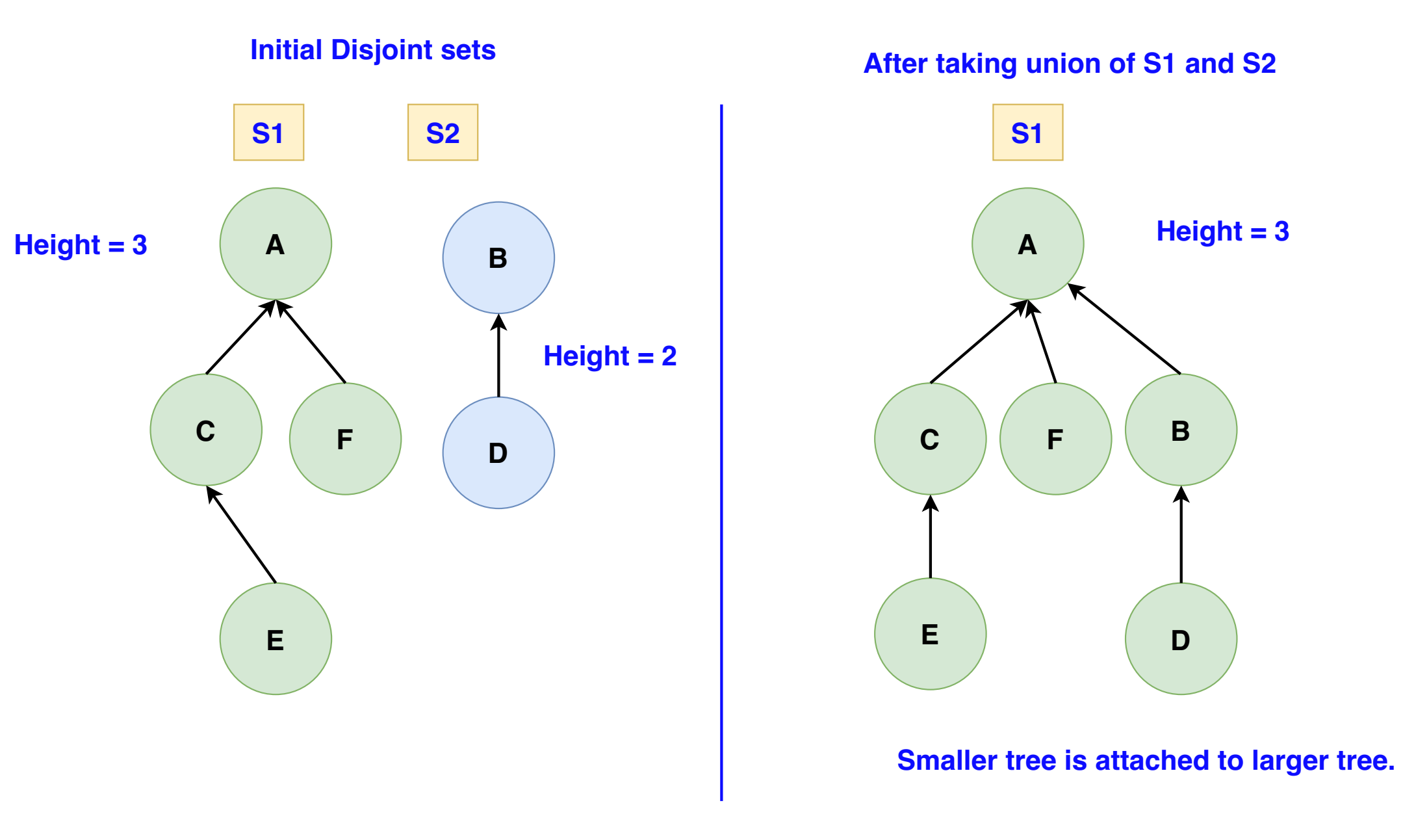
Java

```java
public void unionByRank(int i, int j) {
    // Find the representatives of the sets that i and j are part of.
    int irep = find(i);
    int jrep = find(j);

    // If i and j are already in the same set, nothing to do.
    if (irep == jrep) {
        return;
    }

    // Attach the tree with the smaller rank to the tree with the larger rank.
    if (rank[irep] < rank[jrep]) {
        parent[irep] = jrep;
    } else if (rank[irep] > rank[jrep]) {
        parent[jrep] = irep;
    } else {
        // If ranks are the same, make one the parent of the other and increase its rank
        parent[irep] = jrep;
        rank[jrep]++;
    }
}
```

This function uses the rank array to decide which tree gets attached under which tree.

3. Union by Size:

Similar in spirit to Union by Rank, this optimization considers the size (or number of nodes) of the trees when merging them. The primary goal is to ensure that the smaller tree (in terms of nodes) is always attached to the larger tree. This approach helps in maintaining a balanced tree structure, ensuring that operations on the data structure remain efficient.


Initial Disjoint sets
S1
S2
Height = 3
A
B
Height = 2
C
F
D
E
After taking union of S1 and S2
S1
A
Height = 3
C
F
B
E
D
Smaller tree is attached to larger tree.

Java

```java
public void unionBySize(int i, int j) {
    // Find the root of each set
    int irep = find(i);
    int jrep = find(j);

    // If both elements are in the same set, exit the method
    if (irep == jrep) {
        return;
    }

    // Attach the smaller set to the larger one
    if (size[irep] < size[jrep]) {
        parent[irep] = jrep; // Set jrep as the new parent of irep
        size[jrep] += size[irep]; // Increase the size of the set that jrep is a represe
    } else {
        parent[jrep] = irep; // Set irep as the new parent of jrep
        size[irep] += size[jrep]; // Increase the size of the set that irep is a represe
    }
}
```

This function uses the size array to ensure that the tree with fewer nodes is added under the tree with more nodes.

Example

Here's a complete implementation of the disjoint set with path compression and union by rank:

Java

```java
public class Solution {
    private int[] Rank;
    private int[] Parent;
    private int N;

    // Constructor to create and initialize sets of n items
    public Solution(int n) {
        Rank = new int[n];
        Parent = new int[n];
        this.N = n;
        makeSet();
    }

    // Creates n single item sets
    public void makeSet() {
        for (int i = 0; i < N; i++) {
            // Initially, all elements are in their own set.
            Parent[i] = i;
            // All ranks are initially 1.
            Rank[i] = 1;
        }
```

Run Code Result Stdout

You must run your code first

Execute

Pros and Cons of DSU

- **Pros:**
  - *Efficiency*: Provides near constant time operations for union and find operations, making it highly efficient.
  - *Simplicity*: Once set up, it's straightforward to use for solving problems related to disjoint sets.
- **Cons:**
  - *Space Overhead*: Requires additional space to store the parent and rank of each element.
  - *Initial Setup*: Requires initial setup to create and initialize the data structure.

Why Choose Union-Find Over BFS/DFS?

You can solve similar problems using the Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms, but the Union Find algorithm provides the optimal solution over them. For problems related to connectivity checks and component identification, Union-Find is often more efficient than BFS/DFS.

- **Time Complexity:**
  - **BFS/DFS:** For BFS and DFS, the time complexity is $O(V+E)$, where V is the number of vertices and E is the number of edges. This can be quite slow for large graphs.
  - **Union-Find:** With path compression and union by rank, the amortized time complexity for union and find operations can be approximated as $O(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function. This function grows very slowly, making Union-Find operations almost constant time in practice.

Let's jump onto our first problem and apply the **Union Find** pattern.

← Previous
Solution: Problem Challenge 2: Minimum Height Trees
Next →
Redundant Connection (Medium)
☐ Mark as Completed